



CSI142 Object-Oriented Programming

Methods and Modularity II

Week 3 • Lecture 8

method overloading • scope • shadowing • static context

Bridge: writing clear APIs before classes

Wed 11 Feb 2026 • 08:00–09:00 • Room 252-008



Q1: What is the difference between a parameter and an argument?

Q2: If we call `increment(x)` where `x` is an `int`, can the method change `x` in main?

Q3: If we pass an `int[]` array to a method, can the method change the array contents?



- **Define method overloading and explain why it is useful**
- **Apply the rules: same name, different parameter list (not just return type)**
- **Predict which overloaded method Java chooses in common cases**
- **Explain scope, shadowing, and how to avoid confusion**
- **Recognise static-context limitations (preview for objects)**



Overloading (this week)

- Same method name
- Different parameter list
- Usually same class
- Chosen at compile time

Overriding (later in the course)

- Same method signature
- Different class in inheritance chain
- Uses `@Override`
- Chosen at run time



- ***Overloading*** = same name, different parameter list
- ***Different parameter list means:*** different number of parameters OR different types OR different order
- You cannot overload by changing only the return type
- Use overloading to offer the same idea for different input types



Example: sum overloaded

CSI142 • Semester II 2025/26

```
1 static int sum(int a, int b) {  
2     return a + b;  
3 }  
4  
5 static double sum(double a, double b) {  
6     return a + b;  
7 }  
8  
9 public static void main(String[] args) {  
10     System.out.println(sum(2, 3));  
11     System.out.println(sum(2.5, 4.3));  
12 }
```

Key points

- Same name, different parameter types
- Call site stays simple: `sum(...)`
- Return type follows the chosen overload



- Step 1: look for an exact match (same types)
- Step 2: if needed, use widening ($\text{int} \rightarrow \text{long} \rightarrow \text{double}$)
- Step 3: then boxing ($\text{int} \rightarrow \text{Integer}$) and unboxing
- Step 4: varargs is the last resort
- If two choices are equally good, the call is ambiguous



Pitfall: ambiguous overload

CSI142 • Semester II 2025/26

```
1 static void f(int a, long b) {
2     System.out.println("int,long");
3 }
4
5 static void f(long a, int b) {
6     System.out.println("long,int");
7 }
8
9 public static void main(String[] args) {
10     // f(1, 1);    // ambiguous
11     f(1, 1L);      // choose int,long
12     f(1L, 1);      // choose long,int
13 }
```

Key points

- Ambiguity happens with multiple valid conversions
- Fix by casting or using explicit literals
- Prefer simpler overload sets



- **Local variables exist inside the nearest { } block**
- **Parameters are local to the method**
- **Class fields exist for the lifetime of the object (preview)**
- ***Prefer small scope:* declare variables near first use**
- **If two variables share a name, the nearest scope wins (shadowing)**



```
1 public class Counter {
2     private int count = 0;
3
4     public void demo() {
5         int count = 10;    // shadows the field
6         count++;           // changes the local variable
7         this.count++;      // changes the field
8         System.out.println(count + ", " + this.count);
9     }
10 }
```

Key points:

- Local count hides the field count;
- this.count refers to the field;
- Avoid shadowing unless necessary



static method

- Belongs to the class
- Can be called without an object
- Cannot directly use instance fields

instance method

- Belongs to an object
- Called on an object: `obj.method()`
- Can use fields of that object



Quick practice (2–3 minutes): Which overload is called?

Given overloads: `sum(int,int)` and `sum(double,double)`

- A) `sum(1, 2)`**
- B) `sum(1.0, 2.0)`**
- C) `sum(1, 2.0)`**
- D) `sum(1.0, 2)`**

Write your prediction, then we will discuss.



- ***Overloading*** = same name, different parameter list
- Java chooses overloads at compile time, ambiguity is possible
- Scope rules prevent name confusion, avoid shadowing
- Lab connection: build utility methods and avoid hidden side effects
- Friday: refactoring patterns and the bridge to classes